phase_1_complete_documentation

Phase 1: Foundation and Data Generator (Fleet Telematics Lakehouse)

Project: Fleet Telematics Lakehouse on Microsoft Fabric | SIC 8 / 2026 | Big Data Engineering, Project 3
Last updated: 2026-10-26

This is the single, up-to-date document for Phase 1. It covers the Fabric setup, the generator, the reference data, the
injected anomalies and faults, how to run everything, and how it was verified.

**Status: P1-1 to P1-11 are DONE and verified locally. P1-12 (offline windows) is implemented and switched off by
default; its local verification is pending (section 11.6). The Fabric-side checks in section 13 still have to be filled in.**

---

0. READ THIS FIRST: the 5,000-vehicle reference data is generated LAST

> The full reference data (5,000 vehicles, "evidence" mode) must NOT be generated now. It is generated at the very END
> of the project, immediately before the final evidence run.

Until then the project works only with the **smoke** dataset (20 vehicles) and `config.yaml` stays on `mode: "smoke"`.

0.1 How the mode controls the reference data

`generate_reference_data.py` has no `--mode` option. It reads the mode only from `config.yaml`:

`mode` in `config.yaml`	`python generate_reference_data.py` produces
`"smoke"` (current)	the reference set for the smoke test (20 vehicles)
`"evidence"` (final step only)	the full reference set (5,000 vehicles, about 2,500 peak active)

Only one line of `config.yaml` changes between the two. Do not replace the file, edit that line.

0.2 Final-step procedure (do this LAST)

Start only when ALL of the following are true:

- the pipeline works end to end on the smoke data,
- the `-mode clean` run passes (no alerts, no DLQ rows),
- the acceleration factor `A` has been decided by the team,

- there are at least 2 days left before the trial ends (about 2026-11-23), because the run itself is long.

Steps:

1. **Keep the smoke results.** Rename the local `reference/` to `reference_smoke/` and `evidence/` to `evidence_smoke/`.
2. **Edit one line** in `config.yaml`: `mode: "evidence"` (adjust `acceleration_factor` in the evidence profile if needed).
   Keep `reference_file_faults: false`.
3. **Generate the full reference data:** `python generate_reference_data.py`.
4. **Upload to Fabric:** delete the old smoke files in `Files/reference`, then upload the new `reference/` folder.
5. **Reset Bronze:** stop the generator, then clear every raw table in the KQL database (`.clear table bronze_gps_raw data`, and the same for `bronze_engine_raw`, `bronze_fuel_raw`, `bronze_temperature_raw`, `bronze_device_status_raw`). Confirm each with `| count` = 0.
6. **Run the stream generator:** `python generate_stream.py`. It takes the mode from `config.yaml`. The machine must stay
   on for the whole run: about 16 hours at A = 1, or about 8 hours at A = 2.
7. **Verify:** compare the rows of the raw tables with `messages_sent` in `run_summary.json` (section 15). Upload the
   evidence files (section 14.3).

0.3 Two rules that must never be broken

1. The `mode` in `config.yaml` must match the reference data currently in Fabric at the time of any run.
2. If `simulation.days` is changed, the reference data must be regenerated and re-uploaded before the next run.

---

1. Goal of the phase

Create the world the pipeline will process: the Fabric workspace, all reference data, a reproducible truck simulation that
streams the signals, the injected problems with their ground truth, and a shared library. First verify how the Eventstream
behaves (Day-1 checks of the project guide) so everything else is built on confirmed facts.

---

2. Task status and verification

ID	Task	Status	Evidence
P1-1	Workspace, Lakehouse, Eventhouse, folders, repo	Done	Git repo found and `.env` ignored (audit)

ID	Task	Status	Evidence
P1-2	Day-1 checks	Done	Section 4
P1-3	Config and run modes	Done	Section 6
P1-4	Reference data	Done (smoke size)	Audit: all columns, no overlaps, cold cargo rules
P1-5	Truck simulation, four signals	Done	Section 12
P1-6	Three core anomalies and ground truth	Done	Section 12
P1-7	Vehicle change files for SCD	Done	Audit: V002, V007, V010 found
P1-8	Shared library v1.0	Done, tested on Fabric	Section 8
P1-9	README	This document, saved as README.md	Audit check clears when README.md exists
P1-10	Data-fault injection	Done, verified locally	Section 12
P1-11	Late-arrival injection	Done, verified locally	Section 12
P1-12	Offline windows	Implemented, off by default, verification pending	Section 11.6

3. Fabric setup

3.1 Workspace

- Workspace `SIC8-Fleet-Telematics`. The trial capacity ends about 2026-11-23; the whole plan must finish before that.

3.2 Lakehouse `lh_fleet`

Item	Content
Files/reference	reference CSVs (vehicles, drivers, routes, ...)
Files/landing/offline	bulk files of offline windows (P1-12)
Files/evidence	run evidence (counts, logs, screenshots)

3.3 Eventhouse `eh_fleet` and the Eventstreams

One Eventstream per signal. Each has one custom endpoint source (Kafka protocol) and one Eventhouse destination in
Direct ingestion mode that writes the received text into a raw table. No operators.

Signal	Eventstream	Raw table (KQL database `eh_fleet`)
gps	`es_gps`	`bronze_gps_raw`
engine	`es_engine`	`bronze_engine_raw`
fuel	`es_fuel`	`bronze_fuel_raw`

Signal	Eventstream	Raw table (KQL database `eh_fleet`)
temperature	`es_temperature`	`bronze_temperature_raw`
device_status	`es_device_status`	`bronze_device_status_raw` (empty until P1-12 is on)

Each raw table has exactly one column, `raw_value: string`, that holds the message exactly as received, including broken
messages. The data format of the destination is **TXT**: a typed format (JSON) would drop truncated messages and would
convert wrong-type values, which would hide the injected faults. Create the tables before the wizard:

```
.create-merge table bronze_gps_raw (raw_value: string)
```

(and the same for the other four tables). In the wizard choose "existing table" and format TXT. If the wizard adds a
`data` column, look at `bronze_gps_raw | take 5`: if the text is in `raw_value` run
`.drop column bronze_gps_raw.data`; if the text is in `data` run `.drop column bronze_gps_raw.raw_value` and
`.rename column bronze_gps_raw.data to raw_value`. `getschema` must show only `raw_value`.

3.4 `.env`

One block per signal; the values come from the Event Hub tab of each custom endpoint, SAS Key Authentication page
(connection string = Connection string-primary key; topic = Event hub name = the `EntityPath=` value; bootstrap = the host
of the connection string + `:9093`). Never commit `.env`, never paste keys into chats or notebooks, and regenerate a key
that was exposed.

```
KAFKA_GPS_BOOTSTRAP=<namespace>.servicebus.windows.net:9093
KAFKA_GPS_TOPIC=<topic>
KAFKA_GPS_CONN_STR=<connection string, one line>
# the same three keys for ENGINE, FUEL, TEMPERATURE and DEVICE_STATUS
```

3.5 Test scripts

- `test_all.py`: sends valid test messages (event_id starts with `TEST-ALL-`). Options: `-only <signal>`, `-count N`.
  It warns when BOOTSTRAP, TOPIC and CONN_STR do not match each other or when two signals share a connection string.
- `test_malformed.py`: sends a valid control message (`BAD-TEST-CONTROL-001`) and a truncated one (`BAD-TEST-001`).
- `load_test.py`: Eventstream throughput test (section 4.4).

4. Day-1 tests and findings (P1-2)

4.1 Lakehouse destination test

Check	Result
Can one Eventstream carry all signals?	Yes (separable through `signal_type`)
Schema behaviour	Fixed from the first message; measurements of engine, fuel, temperature and device_status were lost
Raw text column	No
Stream offset	No (only the partition)
Arrival timestamp	`EventEnqueuedUtcTime` (closest equivalent)
Malformed message	Silently dropped, the original text cannot be recovered

Conclusion: the Lakehouse destination cannot be Bronze. A Spark raw-sink notebook was prepared first, but a permanently
running Spark session used the trial capacity (HTTP 430 `TooManyRequestsForCapacity`), and the notebook was never tested
with real data. It is not part of the final design.

4.2 Decision: raw tables in Eventhouse

Bronze is one raw table per signal in Eventhouse (section 3.3), fed by direct ingestion in TXT format. Spark is used only
for the batch jobs that read Bronze. Known limits of this choice, to be accepted by the team:

Item	Effect
Stream offset and `source_stream` columns	Not available in the raw table. The signal is known from the table; offsets are not kept
Arrival time	Not available as a column; use `ingestion_time()` in KQL
Ingestion delay	Direct ingestion is batched; rows can appear a few minutes late
OneLake (Delta) copy of the raw tables	Can lag behind the KQL table; use KQL for exact counts

4.3 Malformed-message test in this design

The test must pass before the evidence run: see section 13, check 2.

4.4 Throughput test (Eventstream ingest)

Script `load_test.py`: 2 producer processes, 30 seconds per step, message mix of the evidence profile. A step passes if the
delivered rate is at least 95% of target, errors are 0 and p95 delivery latency is at most 2 seconds.

Target msgs/s	Delivered/s	p95 latency (s)	Result
200	200	0.14	PASS
500	500	0.64	PASS
1,000	976	5.59	FAIL (latency)
2,000	957	15.34	FAIL
3,000	1,191	46.36	FAIL

R = 500 msgs/s, practical ceiling about 1,000. Limits: measured on the producer side, Eventstream ingest only, before
the move to Eventhouse. It should be repeated on the current setup before the evidence run.

4.5 Acceleration factor

`A <= (0.5 * R) / (peak active trucks * msgs per truck per simulated second)`, with 0.085 messages per truck per second in
the evidence profile: `A <= (0.5 * 500) / (2,500 * 0.085) = 1.18`, so **A = 1**.

Profile	Proposed A	Max A with R = 500
Smoke (20 vehicles)	60	52
Live demo (200)	30	5
Evidence (5,000, 2,500 peak)	10	1

Options for the evidence run: lower A (6 simulated hours take 6 real hours at A = 1), stagger trips over 12 simulated hours
(halves the peak, already set in `config.yaml`), or relax the latency criterion (R about 1,000 allows A about 2.3).
A is NOT final. It stays a value in `config.yaml` until the team decides.

4.6 Day-1 checks status

Check	Status
One Eventstream with several endpoints keeps signals separable	Confirmed
Raw sink keeps raw text, offsets, arrival time	Raw text yes; offsets and arrival time no (section 4.2)
Throughput	R = 500 msgs/s, ceiling about 1,000 (to repeat)
Malformed message kept in the raw table	Section 13, check 2
Power BI refresh / Direct Lake lag	Not tested yet
Airflow Job	Confirmed available in the workspace

5. Project files and their roles

File	Role
`config.yaml`	Single configuration file: seed, mode, intervals, acceleration factor, anomaly, fault and offline settings
`generate_reference_data.py`	Reference CSVs. Mode from `config.yaml` only. With `reference_file_faults: true` it also writes `reference_faults/`
`generate_stream.py`	Truck simulation (generator v1.3). Sends to the Eventstream endpoints and writes the evidence files
`verify_stream.py`	Checks `stream_messages.jsonl` (and the bulk files) against the ground truth. Needs `--sink jsonl`
`audit_phase1.py`	Checks the project folder against the Phase 1 plan
`config.py`, `rng.py`	Config loader, seeded random helper
`test_all.py`, `test_malformed.py`, `load_test.py`	Eventstream tests (section 3.5, 4.4)
`fleet_telematics_shared-1.0.0-py3-none-any.whl`	Shared library, imported as `fleet_shared`

`config.yaml` is replaced as a whole only when a new feature adds keys; otherwise edit single lines (mainly `mode`).

6. Run modes and config

Mode	Vehicles	Routes	Intervals gps/engine/fuel/temp (s)	A	Use
`smoke`	20	3	10 / 10 / 60 / 30	50	End-to-end check (current mode)
`live_demo`	200	8	10 / 10 / 60 / 30	5	Show alerts live
`evidence`	5,000	12	30 / 30 / 120 / 60	1	Scale proof (about 5.4 million messages). Done last (section 0)
`clean`	20	3	10 / 10 / 60 / 30	50	Baseline: no anomalies, no faults, no late trips, no offline windows

A is the acceleration factor: one wall-clock second equals A simulated seconds. `simulation.days` (default 1) gives one trip
per vehicle per day; at A = 50 a simulated day takes about 30 minutes.

6.1 Config keys

Key	Meaning
`seed`	Seed of every random draw
`mode`, `acceleration_factor`, `profiles.<mode>.*`	Active profile and its fleet size, routes, intervals, A, `zero_rates`
`simulation.start`, `departure_window_start_hour`, `days`	Simulated start, first departure hour, number of days
`refrigerated_share`, `driver_ratio`, `fuel_prices`	Fleet composition and declared fuel price
`scd.*`	Hour of the scripted vehicle changes and the bulk change file; bulk counts per 5,000 vehicles
`injection.min_per_type`	Smallest number of each injected case in a run (ignored in clean)
`rates.cold_chain_breach`, `idling`, `route_deviation`	Shares of trips; one third of the breaches are short blips; a GPS wander is added at the deviation rate
`rates.late_arrival`, `late_within_grace`	Shares of trips (section 10)
`rates.<data fault keys>`, `unit_error_vehicle_share`, `sensor_fault_trips`, `stuck_sensor_trips`, `poison_batch_mode`	Data faults (section 9)
`poison_batch.*`	Malformed rate, start hour and length of the poison batch
`reference_file_faults`	`true` also writes `reference_faults/`
`offline.*`	Offline windows (section 11)
`business_rules`	Thresholds written to `reference/business_rules.csv`

7. Reference data, simulation, ground truth (P1-4 to P1-7)

- **Reference data:** routes, checkpoints (15 to 25 per route), cargo types, fuel prices, `vehicles_initial.csv`, drivers,
  trips and `business_rules.csv`, all from the same config. Smoke size: 20 vehicles, 13 drivers, 3 routes, 59 checkpoints,
  4 cargo types, 21 trips, 32 rules. Trips do not overlap per vehicle or driver; cold cargo only on refrigeration-capable
  vehicle versions.
- **Simulation:** one state machine per truck on a simulated clock, sequence numbers per device and signal, messages keyed
  by `vehicle_id`.
- **Anomalies and ground truth:** cold-chain breach, excessive idling and route deviation, with negative cases (cold-chain
  blip, short stops, GPS wander). The plan is written to `evidence/injected_events_log.csv` and `evidence/run_manifest.json` before anything is sent.

- **Vehicle changes (SCD):** `vehicles_changes_YYYYMMDD_HHMM.csv`: V002 retrofit, V007 registration correction, V010 device
  swap; bulk changes appear in larger profiles.

7.1 Ground-truth types (`details` holds `negative_case` and `expected_alert`)

Type	Expected alert	Meaning
`COLD_CHAIN_BREACH`	yes	Temperature above the cargo maximum longer than the alert hold
`COLD_CHAIN_BLIP`	no	Above the maximum, but clearly shorter than the alert hold
`EXCESSIVE_IDLING`	yes	Engine on, truck stopped, 15 to 60 minutes
`SHORT_STOP`	no	Normal 1 to 4 minute stop
`ROUTE_DEVIATION`	yes	GPS 1 to 5 km off the route for 5 to 20 minutes
`GPS_WANDER`	no	Wander of 100 to 350 m, inside the allowed deviation
`LATE_ARRIVAL`, `LATE_WITHIN_GRACE`	yes / no	Section 10
`OFFLINE_WINDOW`, `BUFFER_OVERFLOW`	yes	Section 11
Data faults, `SENSOR_FAULT`, `STUCK_SENSOR`, `UNIT_ERROR`, `POISON_BATCH`	see section 9	Section 9

8. Shared library `fleet_shared` v1.0.0 (P1-8)

Wheel `fleet_telematics_shared-1.0.0-py3-none-any.whl`, import name `fleet_shared`, no dependencies.

Module	Content
`schema`	Schema v1 for the five signals
`rules`	Default rules (same as `business_rules.csv`) and a loader
`validation`	`validate_message()` with the error codes `PARSE_ERROR`, `MISSING_REQUIRED_FIELD`, `TYPE_MISMATCH`, `OUT_OF_RANGE`, `INVALID_GPS`, `INVALID_TIMESTAMP`, `UNKNOWN_VEHICLE`, `UNKNOWN_TRIP`; Fahrenheit to Celsius; flags such as `SENSOR_NULL`; spike, flatline and GPS-jump helpers
`dedup`	`event_id` key, near-duplicate key, winning-copy rule
`geo`	Haversine distance and distance from the route path

Tests: 22 passed, 1 skipped locally (PySpark). Installed in the Fabric Environment `env_fleet`; a notebook attached to it
printed `1.0.0` and `PARSE_ERROR` (keep the screenshot). A shared-library check of a message file:
`python -m fleet_shared.selfcheck evidence/stream_messages.jsonl reference/business_rules.csv`.

9. P1-10: Data-fault injection

Faults are applied to single messages after they are built (the world is fine, the data is wrong). Sensor faults and unit
errors are planned per trip. Every fault is written to `injected_events_log.csv` BEFORE anything is sent, with the
`event_id` of the affected message and `details` (`device_id`, `sequence_number`, `handling`, `expected_error_code`, and
`expect` = the field values that must appear in the message, `"__missing__"` for a removed field).

How it stays reproducible: data faults use their own random stream per vehicle, so the physical anomalies are identical
with or without faults; a dry pass collects the faults before sending; at most one fault per message; `clean` mode injects
nothing.

injection_type	Default rate	Signals	What is sent	Expected handling
DROPPED_MESSAGE	0.3%	all four	Built and numbered, never sent (sequence gap)	Gap, completeness, DATA_GAP
DUPLICATE	0.5%	all four	Same message twice	Removed in Silver (event_id)
NEAR_DUPLICATE	0.2%	all four	Same vehicle, signal, time and sequence number, new event_id	Composite-key rule
MALFORMED	0.2%	all four	JSON cut short	DLQ: PARSE_ERROR
MISSING_FIELD	0.2%	all four	vehicle_id, event_time or trip_id removed	DLQ: MISSING_REQUIRED_FIELD
WRONG_TYPE	0.2%	all four	One numeric field as text	DLQ: TYPE_MISMATCH
INVALID_GPS	0.1%	gps	(0,0) or latitude above 90	DLQ: INVALID_GPS
IMPOSSIBLE_VALUE	0.1%	gps, engine	speed 400 km/h or negative RPM	DLQ: OUT_OF_RANGE
BAD_TIMESTAMP	0.1%	all four	Future date, dd/mm/yyyy format, or no timezone	DLQ: INVALID_TIMESTAMP
UNKNOWN_VEHICLE	0.05%	all four	vehicle_id `VUNKnnnn`	DLQ: UNKNOWN_VEHICLE
SCHEMA_DRIFT	0.05%	all four	Half: extra field `firmware_version`; half: `schema_version` 2	Extra field kept and flagged; version 2 to DLQ
TEMPERATURE_SPIKE	0.1%	temperature	One reading +10 to +15 C	Kept, flagged SPIKE

injection_type	Default rate	Signals	What is sent	Expected handling
UNIT_ERROR	1% of refrigerated vehicles	temperature	All cold trips report F (`temperature_unit` = "F")	Converted to C, flagged UNIT_CONVERTED
SENSOR_FAULT	2% of cold trips	temperature	5 to 30 min of null readings, `sensor_status` FAULT	Kept, flagged SENSOR_NULL, alert
STUCK_SENSOR	0.5% of cold trips	temperature	Frozen reading, longer than `flatline_readings`	Kept, flagged FLATLINE (no alert expected)
POISON_BATCH	off	all	Malformed rate 10% for 5 simulated minutes on day 1	HIGH_INVALID_RATE alert, gate 1 stops Gold

About 1% of messages are invalid, below the 2% alert threshold and the 5% gate limit. `injection.min_per_type` guarantees
at least one sensor fault, stuck sensor and unit-error vehicle in the smoke run; per-message faults are random.

9.1 Reference-file faults

With `reference_file_faults: true`, `generate_reference_data.py` also writes `reference_faults/` (the normal `reference/`
stays valid). Upload it to a separate folder (for example `Files/reference_faults`), never to `Files/reference`. It is meant
for testing the file-ingestion step (DLQ stage FILE_INGEST).

File	Fault
`trips.csv`	An extra trip overlapping a trip of the same vehicle; an extra cold-cargo trip on a vehicle that is never refrigeration-capable
`cargo_types.csv`	Cargo `BAD_RANGE` with minimum 10 above maximum 5
`routes.csv`	Row `R999` with an extra delimiter
`drivers.csv`	Column `license_category` removed
`reference_faults_log.csv`	Ground truth of these faults

Set the flag back to `false` after generating.

10. P1-11: Late-arrival injection

Case	Config key	What happens	Expected alert
Slow trip	`rates.late_arrival` (5% of trips)	Arrives 10 to 60 minutes after planned end plus grace	Yes (`LATE_ARRIVAL`, cause `INJECTED_SLOW_TRIP`)
Near-miss	`rates.late_within_grace` (5%)	Arrives after the planned end, inside the grace	No (`LATE_WITHIN_GRACE`)

- The truck **paces itself** to a chosen arrival time instead of using a fixed slowdown, because a fixed 60 to 80% speed
  cannot hit a 10 to 60 minute window on routes of very different length. No extra stops are added.
- The ground truth comes from the **real arrival times** of the dry pass: every trip that really arrives later than planned
  end plus grace is logged as `LATE_ARRIVAL`; trips that arrive late as a side effect of idling get cause `SIDE_EFFECT`.
- A late trip must end before the next trip of the same vehicle starts; trips without room are skipped. Drivers are not
  checked (no message carries a driver).
- Own random stream: the other anomalies and faults are unchanged.

---

11. P1-12: Offline windows (store-and-forward)

Implemented in generator v1.3. **Off by default** (`offline.enabled: false`): with it off the output is identical to the
run without P1-12.

11.1 Behaviour

While offline the device keeps measuring and stores each reading with its true `event_time` and consecutive sequence
numbers. On reconnect it sends a `device_status` report (`RECONNECTED`, `offline_from`, `reconnected_at`, `stored_*`,
`dropped_messages`, `upload_method`, `upload_id`), then the stored readings, either as a burst on the stream (`STREAM_BURST`) or as a bulk file (`BULK_FILE`).

Rule	Value
Share of trips	8% (`offline.trips_share`)
Outage length (buffer survives)	3 to 60 minutes, cut to 95% of the buffer if shorter (`min_minutes`, `max_minutes`)
Buffer survives / overflows	85% / 15% (`buffer_survives_share`)
Buffer capacity	300 messages per signal (`buffer_capacity_messages`); the oldest are lost on overflow
Overflow window	(capacity x gps interval) + 5 to 20 minutes (`overflow_extra_min/max_minutes`)
Burst or file	Average burst share 0.5 (`burst_share`); longer outages favour the file

An overflow window must be long enough to overflow: with the guide's 3 to 60 minute windows and 30 s intervals the
evidence profile stores only 120 messages per hour per signal, so a buffer of 300 could never overflow.

11.2 Bulk files

`<out>/landing/offline/<vehicle_id>_<upload_id>.jsonl` (for example `V009_UPL00001.jsonl`), one JSON message per line,
same schemas as the stream. Upload the files to `Files/landing/offline/`. The files are not part of `stream_messages.jsonl`. The generator removes old `*.jsonl` files of that folder at the start of a run.

11.3 Ground truth

`OFFLINE_WINDOW` (start/end of the outage, `event_id` of the report, `upload_method`, `upload_id`, `file_name`, `built_by_signal`, `stored_by_signal`, `dropped_total`, `expected_episode_status` CLOSED or CLOSED_WITH_LOSS, expected alert
`DEVICE_OFFLINE`) and, when readings were lost, `BUFFER_OVERFLOW` (`lost_ranges` with device, signal, first and last
sequence number; expected alert `DATA_GAP`).

11.4 Limits

- No data faults (P1-10) are applied to stored readings, otherwise a dropped message would look like loss in transit.
- Offline windows are placed where no other planned anomaly is, so they never hide a breach or a deviation.
- Needs the `device_status` endpoint in `.env`; the generator stops with a message if it is missing.
- How the bulk files are loaded into Bronze (file ingestion, `ops_ingestion_log`) is a pipeline task, not a generator task.

11.5 Reconciliation with offline windows

Signal messages built = stream signal rows + bulk-file rows + rows lost by buffer overflow + P1-10 dropped − DUPLICATE −
NEAR_DUPLICATE copies. `run_summary.json` gives `device_status_reports`, `bulk_files`, `bulk_file_messages` and
`messages_lost_buffer_overflow`; `messages_sent` includes the device_status reports and the bursts.

11.6 Verification checklist (fill in)

- [ ] `enabled: false`: smoke run still gives 74,194 messages, `PROBLEMS: none`, determinism `True`
- [ ] `enabled: true`: `verify_stream.py` prints `PROBLEMS: none`, `OFFLINE_WINDOW n/n`, `BUFFER_OVERFLOW n/n`
- [ ] `enabled: true`: determinism `True` (stream and bulk files)
- [ ] Clean run: 70,024 messages, no offline windows
- [ ] Bulk files uploaded to `Files/landing/offline/` and the `device_status` table receives the reports

---

12. Local verification record (2026-10-26)

Run	Result
Reference data	20 vehicles, 13 drivers, 3 routes, 59 checkpoints, 21 trips, 32 rules, 1 change file
Kafka smoke run, P1-10 code	71,455 sent, 71,455 delivered, 0 errors (gps 29,737, engine 29,774, fuel 4,985, temperature 6,959); built 71,133
Clean run (P1-11 code)	70,024 messages (gps 29,128, engine 29,128, fuel 4,862, temperature 6,906); only `SHORT_STOP` planned; `PROBLEMS: none`
Smoke run (P1-11 code, local)	74,194 messages (gps 30,895, engine 30,926, fuel 5,176, temperature 7,197); 74,036 parsed + 158 unparseable; built 73,866; `PROBLEMS: none`
Anomalies	Breach V011, blip V013, idling V015 and V007, deviation V005, V009, V007, wander V012 and V006: all `OK`
Late arrivals	`LATE_ARRIVAL`: V010 (45.3 min beyond grace), V001 (22.3), V015 (9.2, side effect). `LATE_WITHIN_GRACE`: V002, V019, V017, V020 (9.7 to 22.3 min after the planned end, grace 30): all `OK`
Data faults	All 17 types found as logged (for example DROPPED 190, DUPLICATE 373, MALFORMED 158, MISSING_FIELD 145)
Determinism	Same seed twice gives identical `stream_messages.jsonl` (`True`)
`audit_phase1.py`	28 PASS, 1 FAIL (README.md missing; cleared by saving this document as README.md)

13. Fabric checks to complete

Run in the KQL database `eh_fleet`, each query alone. Clear the tables before a run: `.clear table <name> data` for each of the five raw tables.

1. **Row counts per raw table** after a Kafka run:
   Expected for the P1-10 Kafka run above: gps 29,737, engine 29,774, fuel 4,985, temperature 6,959, total 71,455.
   Expected for a P1-11 Kafka run: gps 30,895, engine 30,926, fuel 5,176, temperature 7,197, total 74,194.
   Result: _____

   ```
   union withsource=T bronze_gps_raw, bronze_engine_raw, bronze_fuel_raw, bronze_temperature_raw, bronze_devi
   ce_status_raw
   | summarize rows = count() by T
   ```

2. **Broken messages are kept** (run `python test_malformed.py`, wait a few minutes):
   Pass: 2 rows (the valid control and the truncated text). Result: _____

   ```
   bronze_gps_raw
   | where raw_value has "BAD-TEST"
   | project raw_value
   ```

3. **Unparseable rows** after a smoke run:
   Pass: the total equals the number of `MALFORMED` rows of `injected_events_log.csv` (158). Result: _____

```
union withsource=T bronze_gps_raw, bronze_engine_raw, bronze_fuel_raw, bronze_temperature_raw, bronze_devi
ce_status_raw
| where gettype(todynamic(raw_value)) != "dictionary"
| summarize unparseable = count() by T
```

4. **Exact duplicates:**
   Pass: rows – distinct_ids equals the number of `DUPLICATE` rows of the log (373 in the P1-11 run).
   Result: _____

```
let ids = union bronze_gps_raw, bronze_engine_raw, bronze_fuel_raw, bronze_temperature_raw, bronze_device_
status_raw
    | extend j = todynamic(raw_value) | where gettype(j) == "dictionary" | project event_id = tostring(j.e
vent_id);
print rows = toscalar(ids | count), distinct_ids = toscalar(ids | distinct event_id | count)
```

---

14. How to run

14.1 Setup

```
pip install pyyaml numpy python-dotenv confluent-kafka
```

`.env` as in section 3.4. `--sink jsonl` and `--sink none` need no Kafka and no `.env`.

14.2 Local run (smoke, no Fabric)

```
python generate_reference_data.py
python generate_stream.py --mode clean --sink jsonl --fast --out evidence_clean
python verify_stream.py evidence_clean reference
python generate_stream.py --sink jsonl --fast
python verify_stream.py evidence reference
python audit_phase1.py --root .
```

Determinism: run twice with `--out run_a` and `--out run_b` and compare the md5 of the two `stream_messages.jsonl`
(and, with offline windows on, of the bulk files). Delete `evidence_clean`, `run_a` and `run_b` afterwards, because the
audit reads the first evidence files it finds.

`generate_stream.py` options: `--mode`, `--accel`, `--sink kafka|jsonl|none`, `--fast`, `--reference DIR`, `--out DIR`.

14.3 Run into Fabric

1. Upload `reference/*` to `Files/reference` (delete the old files first); `reference_faults/` to its own folder.

2. Clear the five raw tables (section 13).
3. `python generate_stream.py` (paced by A; progress every 10 seconds, `lag` should stay near 0, otherwise lower A with `-accel`).
4. Reconcile (section 13 and 15). Do not run the generator twice against the same tables: every run adds its messages again.
5. Upload `run_manifest.json`, `injected_events_log.csv`, `trip_outcomes.csv`, `run_summary.json` to `Files/evidence` (not `stream_messages.jsonl`), and `evidence/landing/offline/*` to `Files/landing/offline/` when offline windows are on.

14.4 Outputs

File	Content
`reference/*.csv`	Reference data and `vehicles_changes_*.csv`
`evidence/run_manifest.json`	run_id, seed, profile, vehicle_count, sim_start, wall_start, acceleration_factor, intervals, rates, offline settings, planned injections
`evidence/injected_events_log.csv`	Ground truth, written before sending, read only by the evaluation step
`evidence/trip_outcomes.csv`	Actual start and end, distance, fuel, idle minutes, minutes late beyond grace
`evidence/run_summary.json`	Messages sent per signal, dropped, built, offline counters, delivery result
`evidence/stream_messages.jsonl`	All messages (only with `--sink jsonl`)
`evidence/landing/offline/*.jsonl`	Bulk files of the offline windows

15. Reconciliation

- Messages built = `messages_sent` − device_status reports + bulk-file rows + overflow-lost rows + `DROPPED_MESSAGE` − `DUPLICATE` − `NEAR_DUPLICATE`.
- Rows in the raw tables (stream) = `messages_sent` of `run_summary.json`, per signal = `per_signal`.
- Parsed rows + unparseable rows = rows of the raw tables; unparseable = `MALFORMED` rows of the log.
- Rows of the raw tables − distinct `event_id` = `DUPLICATE` rows of the log.
- Expected DLQ rows from the stream = MALFORMED + MISSING_FIELD + WRONG_TYPE + INVALID_GPS + IMPOSSIBLE_VALUE + BAD_TIMESTAMP + UNKNOWN_VEHICLE + SCHEMA_DRIFT with `drift_kind` = unknown_version.
- Sequence gaps are explained by `DROPPED_MESSAGE`, `MALFORMED` (delivered, number unreadable) and `BUFFER_OVERFLOW`.
- `trip_outcomes.csv` column `messages` counts built messages, not sent ones; `injection_id` numbers shift when features are switched on or off.

16. Known limits

- `verify_stream.py` loads the whole message file in memory: use it on smoke or `live_demo`, not on evidence.
- The generator is one process; if `lag` grows, lower A.
- The evidence profile has 12 routes (the guide mentions about 40); add city pairs to `ROUTE_DEFS` in `generate_reference_data.py`.
- The generator writes the ground truth of late arrivals from the real arrival times; a truck that arrives late inside the
  grace without being injected is not logged.
- Offline windows: no data faults on stored readings (section 11.4).
- The evidence value of A is not final (section 4.5).

18. Risks and limits

- **Mode mismatch** between `config.yaml` and the reference data in Fabric gives wrong reconciliation. Always check before a run.
- **Keys:** a connection string that was pasted anywhere must be regenerated; `.env` must never be committed.